

FROSTBIT: Compiling Boolean Filters over Compressed Bitmaps into Allocation-Free, Provably-Sized Execution Plans

William Liu
Exa
San Francisco, USA
wliu@exa.ai

ABSTRACT

Compressed bitmaps in the Roaring format are the de facto standard for boolean filtering in search and analytics engines, and a decade of work has produced highly tuned SIMD kernels for intersecting, uniting, and subtracting individual containers. We observe that the remaining performance is not in the kernels: it is in everything an engine does *around* them — allocating and growing result buffers, re-deriving representation decisions per operation, streaming containers that cannot survive the query, and choosing fold orders blind to the memory system.

FROSTBIT is a bitmap engine built on a single organizing principle: a boolean filter expression is *compiled, once, into a complete execution manifest* by an analysis pass that pre-computes every decision the fold would otherwise make — the exact set of output containers per operator, a proven byte ceiling for each (so execution contains no allocation or growth path at all), each key’s accumulator representation, the fold order, and a container-granular liveness mask that skips every 64 KiB block that cannot survive the root of the expression. The executor is deliberately trivial: it streams the provably minimal set of containers through pre-sized, thread-pooled buffers, and its writes *are* the output — results serialize in place, so a steady-state query performs zero heap allocations.

On an end-to-end corpus of 25,000 randomly-shaped filter trees (up to 100 leaves and 15 levels), FROSTBIT evaluates the full corpus 3.4× faster than the standard Roaring library and 2.25× faster than its nightly-only SIMD build, while winning every operation shape in a 2–16-way microbenchmark sweep by 1.2–15×; container-granular pruning alone accelerates selective filters by up to 33× and short-circuit evaluation resolves contradictory filters 2,900× faster. FROSTBIT is the production filtering substrate for all boolean filtering at Exa, a web-scale search engine.

1 INTRODUCTION

Every serving system we know of that filters documents at scale eventually converges on the same data structure: a compressed bitmap in the Roaring format [1], partitioned into per-64Ki-value containers that adaptively choose between sorted arrays, packed bitmaps, and run-length encodings. The container *kernels* — the tight loops that intersect two sorted arrays or AND two 1024-word bitmaps — have been optimized to the point of diminishing returns by a long line of SIMD work [4–8].

Our starting observation, from operating a web-scale search engine, is that production filter latency is not dominated by those kernels. It is dominated by the scaffolding: the allocator invoked for every intermediate result; the representation decisions (array or bitmap? run or not?) re-derived inside every operation; containers faithfully streamed, decoded, and merged even when a cheap

argument shows they cannot contribute to the final result; and fold loops whose memory access patterns defeat the prefetcher at exactly the fan-ins production queries use.

FROSTBIT attacks the scaffolding. The design premise is that a filter expression — an arbitrary tree of AND, OR, and DIFF over frozen bitmaps — should be *analyzed once, completely*, and that the analysis should be strong enough that execution becomes a straight-line replay with no decisions left to make. Concretely, the analysis pass pre-computes:

- **the exact output container set of every operator** in the tree, with a *proven* byte ceiling for each output slot. The executor therefore contains no reallocation or growth path — not as an optimization, but as an invariant the planner discharges. We refer to these ceilings as *capacity clamps* (§4);
- **every representation decision**: which keys accumulate as arrays, bitmaps, or native run containers, including a fan-in-aware promotion rule derived from a calibrated cost model rather than a fixed cardinality threshold (§5);
- **the fold order of each operator** — key-major or partner-major — selected by comparing the plan’s proven accumulator footprint against the cache, because the two orders are memory-optimal in disjoint regimes (§6);
- **a container-granular liveness mask** (“hole-punching”): an upward pass computes each subtree’s surviving key set, and every cursor in the plan skips dead 64 KiB blocks before a single byte of them is streamed. Together with subtree short-circuit guards compiled into the plan, the executor streams the provably minimal container set at the same granularity Roaring itself uses (§7);
- **all working memory**: every buffer the fold will touch is sized from the plan and drawn from per-thread pools; results serialize in place inside the working arena, so the fold’s writes are the output’s bytes and a steady-state query never calls the allocator (§8).

None of the individual kernels in FROSTBIT is novel — deliberately so; we verified against a systematic survey of the literature (310 collected papers) that our merge kernels match the known state of the art, and we show where measurement told us to deviate from it. The contribution is the layer above: to our knowledge no published system compiles boolean container-tree queries into allocation-free, capacity-proven, liveness-pruned execution manifests, and no published system selects fold order from a static memory footprint proof. The result is an engine that starts from parity with the best-tuned kernels and then wins everywhere the scaffolding used to be.

This paper makes the following contributions:

- (1) the plan-as-manifest architecture: single-pass, construction-fused analysis producing complete execution manifests with proven capacity clamps, reusable across queries (§4);
- (2) cost-model dispatch at three levels — kernel tier, representation promotion, and fold order — each calibrated and ablated on measured constants, including a fan-in-aware union promotion rule and, we believe, the first cache-footprint-driven fold-order selection (§5, §6);
- (3) automatic container-granular pruning and compiled short-circuit guards, which reduce streamed bytes to the provable minimum at container granularity (§7);
- (4) an allocation-free execution discipline, including in-place serialization where the arena buffer becomes the result (§8);
- (5) an evaluation against both distributed configurations of the standard Rust Roaring library, including a 25,000-tree corpus, complete per-technique ablations with negative results, and a per-tree audit methodology (§9);
- (6) deployment experience as the filtering substrate of a production web-scale search engine (§10).

2 BACKGROUND AND RELATED WORK

Compressed bitmaps. Word-aligned RLE schemes (BBC, WAH and its descendants PLWAH, COMPAX, VAL-WAH) [3] trade decode cost for space; Roaring [1, 2] replaced them in most modern systems with a two-level design — a u16-keyed directory of per-64Ki containers stored as sorted arrays (≤ 4096 values), 8 KiB bitmaps, or run lists — that supports random access and SIMD kernels. FROSTBIT adopts Roaring’s container model wholesale (and is wire-convertible to it); our frozen format adds a structure-of-arrays directory and 64-byte container alignment so bitmaps can be operated on directly from an mmap.

Set intersection kernels. Sorted-array intersection spans the adaptive lineage [10] and the SIMD lineage: shuffle-based compare-all-pairs [4, 6], branch-reduced merging [5], byte-filtered variants [7], and segment-bitmap prefilters [8]. Cost-model algorithm selection appears per-query in web-search list intersection [9]. FROSTBIT’s kernels are the portable synthesis of this line (compare-all-pairs with both-operand rotation, Inoue-style merge networks for union, table-driven compaction), dispatched per container pair by explicit cost models; our survey of this literature (§9) found no published kernel our measured throughput trails.

Bitmaps in systems. Bitmap and signature indexes power analytics and search engines from Model 204 and FastBit through Druid, Pinot, and BitFunnel [11]; column stores accelerate scans with bit-parallel layouts (BitWeaving, ByteSlice). These systems document the container structures; none, to our knowledge, documents compiling filter *expressions* into pre-proven execution plans, which is precisely the layer FROSTBIT contributes. UpBit and CUBIT address updatability, which FROSTBIT deliberately does not: our bitmaps are frozen artifacts of an indexing pipeline, which is what makes whole-plan static analysis sound.

3 SYSTEM OVERVIEW

A FROSTBIT bitmap is a single contiguous buffer: a 16-byte header, a structure-of-arrays container directory (key, type, cardinality, offset — 8 bytes per container), and a data section in which bitmap

containers sit on 64-byte boundaries. Buffers are validated once at the trust boundary and thereafter read zero-copy, including directly from memory maps. A builder ingests ascending values; conversions to and from Roaring are container transcodes.

Queries are expressions over frozen bitmaps: leaves (borrowed views or shared owned bitmaps) combined by AND, OR, and DIFF combinators. *Constructing the expression is the analysis*: each combinator fuses its children’s plans into a flat post-order program, flattening same-operator chains ($\text{And}(\text{And}(a, b), c)$ becomes one 3-way fold), and computing everything §4 describes. *materialize()* replays the program; the plan is reused across calls, which matches the production pattern of compiled filters evaluated against many segments.

4 THE ANALYSIS PASS

The analysis pass is the core of the system. Its output — the *manifest* — makes three kinds of promises to the executor, and the executor is correct only because all three are discharged statically.

4.1 Shapes and capacity clamps

For every node, bottom-up, the analyzer computes the node’s *shape*: the set of container keys it can emit, and for each key an upper bound on the output’s cardinality, run count, and representation, mirroring exactly the decision ladder the kernels use. Leaf shapes are exact (read from the directory); interior shapes are sound over-approximations. From the shape, each operator gets an arena plan: one slot per possible output container, with a byte capacity *proven sufficient for every state the fold can pass through* — the verbatim-copy case, the shrunk case, the run-split worst case ($2 + 4 \cdot (\text{runs} + \text{subtrahend items})$ bytes), or a full 8 KiB bitmap, whichever the ladder can reach. Slots are claimed by key at execution, so a plan whose key set over-approximates reality (as intersection plans necessarily do) remains correct with zero waste beyond the unclaimed slots.

The consequence is an invariant, not an optimization: *the execution path contains no allocation, growth, or reallocation code at all*. The arena asserts (in debug builds) that every recorded container fits its clamp; an 11,600-case randomized stress suite exercises the proof obligations. When we extended the system with partner-major folding (§6), the one candidate design that would have required loosening a clamp — seeding intersections from the driver rather than the per-key minimum — was rejected for exactly that reason.

4.2 Fan-in-aware analysis

Because same-operator chains flatten, an operator’s true fan-in is not its child count in the source expression. The analyzer threads each spliced child’s *operand weight* through shape computation, so per-key fan-in bounds are exact upper bounds on what the executor will see. This matters because two of the dispatch decisions below are functions of fan-in, and an early version that counted tree children instead of flattened operands produced plans whose kernels disagreed with their clamps — caught immediately by the stress suite, and fixed by making the plan’s clamp the *single authority* for representation choice: the kernel never re-derives a decision the planner already proved.

5 KERNEL AND REPRESENTATION DISPATCH

FROSTBIT mixes algorithms at every level where measurement showed a crossover, and encodes each crossover as an explicit, commented cost model.

Three-tier sorted-array kernels. Balanced pairs ($|B| \leq 4|A|$) take a shuffle-merge: an 8-lane compare-all-pairs using rotations of *both* operands (5 permutations instead of 7), with branchless table-driven output compaction. Moderately skewed pairs take a galloping broadcast-scan (one lane broadcast against a galloped window of the larger side). Heavily skewed pairs take a scalar galloping binary search, but only when its $|A| \log |B|$ work clears the scan’s $|B|/8$ linear advance *by a margin* — binary search mispredicts, so on-paper parity is not enough. Every constant in these predicates was set by measurement, and §9 reports the experiments — including the ones that failed.

Fan-in-aware union promotion. Union accumulators outgrow arrays. The classical rule promotes to a bitmap at a fixed cardinality threshold (256 in CRoaring-derived engines). But the array path’s cost is $\Theta(\text{sum} \cdot (n+1)/2)$ streamed elements at fan-in n (each merge re-streams the accumulator), while the bitmap path is a flat clear + scatter + popcount. Solving with measured constants gives a promotion rule that is a function of both: never at $n \leq 3$, above ~ 1000 summed values at $n=4$, falling as fan-in grows. The fixed threshold cannot be made correct: tuned low it destroys low-fan-in expressions (AND of small OR-groups — we measured $89 \rightarrow 115$ ns on that shape), tuned high it abandons wide unions (we measured 203 ns on a 4-way union the fan-in rule executes in 54.9 ns).

Run containers. Dense runs fold natively — $\text{run} \cap \text{run}$, $\text{run} \cup \text{run}$, and run-splitting difference operate on typed run slices inside bitmap-sized clamps, converting to bitmaps only on proven overflow.

6 FOLD ORDER AND THE MEMORY SYSTEM

An n -way fold visits a cross product of containers: keys $\times$ operands. *Key-major* order (gather all operands at a key, fold, move to the next key) keeps one accumulator cache-hot but resumes n interleaved input streams per key. *Partner-major* order (stream one operand fully, then the next) reads every input as a long sequential stream — the pattern prefetchers are built for — but revisits the whole accumulator set every pass.

Profiling difference folds at fan-in 16 localized a +50% per-block penalty entirely to memory: the kernel ran at 1.45 ns/8-lane-block standalone, 2.17 ns inside a 32-key fold, and 1.56 ns in a single-key control — the interleaved resumption of 16 partner streams was defeating L1. Partner-major folding recovered it, *and then dense unions showed the mirror image*: 16 bitmap accumulators are 128 KiB, revisiting them every pass thrashes L1, and key-major won by 12%.

The resolution is the system’s thesis in miniature: *the plan already knows the answer*. The accumulator footprint is the sum of the proven capacity clamps; if it is cache-resident (≤ 64 KiB) the plan is laid out double-buffered and the fold runs partner-major (array merges flip each slot between two arena sides, so out-of-place kernels need no staging copies); otherwise key-major. Intersection is the deliberate exception: a 1-key vs. 32-key control shows 16-way AND scales linearly with key count (695 ns $\rightarrow$ 21.6 ns, $31.1\times$

for $32\times$) — after the first fold the accumulator is a sliver and partners are galloped into, not re-streamed — so there is no pathology to fix, and partner-major would have required loosening the intersection clamps. Each operator got the fold order the measurements assigned it.

7 STREAMING THE PROVABLE MINIMUM

The analysis pass eliminates work at three granularities, all compiled into the manifest.

Hole-punching (container granularity). An upward pass computes each node’s surviving key set ($\cap$ for AND, $\cup$ for OR, left side for DIFF). When an AND’s intersected key set is provably narrower than some child’s, the planner materializes it as an 8 KiB liveness bitset and every leaf cursor in the plan skips dead keys in place — dead blocks are never streamed, decoded, or merged. The check is free (compare key counts), the mask is derived from the plan’s own shape (a stale or foreign mask is unrepresentable), and pruning is sound because it is monotone: only keys that cannot reach the root are dropped. A selective filter — one narrow 4-block conjunct against 256-block disjunctions — runs in 7.35 ns against 241 ns unpruned and 374 ns for Roaring’s SIMD build. This is, to our knowledge, the first published container-granular analogue of zone-map pruning that operates *across* a boolean expression tree rather than along a scan.

Subtree short-circuits (operator granularity). The compiled program carries guard instructions: if an AND operand (or a DIFF minuend) evaluates empty, the guard drops the partial operands and jumps past the fold — sibling subtrees are never evaluated. Guards use relative offsets so they survive splicing. A contradictory filter (empty subtree beside an expensive disjunction) resolves in 133 ns against 392 ns for Roaring: $2,900\times$, because the expensive branch is simply never run.

Intra-operator short-circuits (element granularity). Intersections fuse population counting into the AND so an accumulator that empties stops the key immediately; per-key folds order partners most-selective-first; per-slot state kills dead keys across partner-major passes.

8 MEMORY DISCIPLINE

Every buffer in the execution path is pre-allocated and pooled per thread: the op arena (slot storage + fixed scratch), the fold’s cursor and reference scratch, the tree evaluator’s operand stack, and — closing the loop — the result buffers themselves. Arenas are laid out with worst-case header and directory room reserved at the front, so serialization *compacts payloads leftward in place* (sources never precede destinations, by the clamp ordering) and hands the arena’s own buffer out as the result; a result-pool buffer is swapped back so the pools circulate. Tree evaluation chains intermediate arenas directly as operator inputs — nothing is serialized until the root.

The consequence is measurable, not rhetorical: after warm-up, a query performs *zero* heap allocations end-to-end, verified by sampling profiles (the standard library’s allocator simply does not appear). Roaring’s evaluator, by contrast, allocates every intermediate; in our profiles roughly 6% of its runtime is allocator time — which means FROSTBIT starts every comparison six points ahead before any algorithmic difference applies, and exhibits none of the

Table 1: 8-way folds, medians (μ s). R = Roaring default, R^{simd} = nightly SIMD build.

	FROSTBIT	R	R ^{simd}
AND (sparse / dense / runs)	17 / 12 / 2.7	73 / 23 / 4.6	25 / 22 / 4.4
OR (sparse / dense / runs)	117 / 21 / 3.0	1200 / 39 / 7.1	789 / 39 / 7.2
DIFF (sparse / dense / runs)	72 / 53 / 2.0	1110 / 158 / 5.0	80 / 155 / 5.1

allocator-induced tail variance that motivated this design in production.

9 EVALUATION

Setup. Criterion, 30 samples $\times$ 4 s per benchmark (corpus: 10 $\times$ 20 s), Apple M-series (aarch64/NEON). Baselines: Rust roaring v0.11.4 in both of its configurations — the default build every stable-toolchain user gets, and the nightly-only simd build (its strongest form). All tree benchmarks time FROSTBIT’s full pipeline including plan construction. A two-pass runner interleaves both baselines into one report. Correctness is enforced throughout by differential testing against Roaring (400-tree random differentials, punched-tree differentials, run containers, 10M-element round trips) plus the 11.6k-case clamp stress.

9.1 Operation sweep

Across the full 2–16-way sweep over sparse-array, dense-bitmap, and run-container regimes, FROSTBIT wins *every cell* against default Roaring by 1.2–15 $\times$ (sparse-array operations by 3.6–15 $\times$: its array kernels are scalar without the nightly feature, while FROSTBIT’s SIMD always runs). Against the nightly SIMD build, FROSTBIT wins everywhere except two 16-way cells inside measurement noise (0.98 $\times$, 0.99 $\times$). Table 1 shows the 8-way column.

9.2 Filter-tree corpus

The headline benchmark is a corpus of 25,000 deterministic random filter trees (295,455 leaves; sizes log-spread 2–100 leaves, depths 2–15; per-tree shape profiles spanning deep AND-chains, flat 100-way ORs, and DIFF-heavy composites; 100 trees pinned to both extremes at once), evaluated end-to-end with per-query plan construction. FROSTBIT clears the corpus in **2.63 s** (9.5K trees/s) against 9.06 s for default Roaring (3.4 $\times$) and 5.92 s for the SIMD build (2.25 $\times$). Named production-like shapes: a conjunction of OR-groups runs 72.6 *ns* vs. 331/102 *ns*; a five-way flattened conjunction 34.1 *ns* vs. 315/80 *ns*; a base-and-negation filter 17.5 *ns* vs. 41/44 *ns*.

A per-tree audit (minimum-of-5 timing of every corpus tree on both engines) is part of our methodology: at its worst point the audit surfaced 1,245 trees >3 *ns* behind the SIMD build; root-causing them produced the hoisted bitmap-probe loop (offenders fell to 743) and ultimately the auto hole-punching pass. We consider the audit-driven loop — measure per-shape, explain the worst offender structurally, fix the class — a reusable methodology, and we report it because each fix is an ablation with a causal story.

9.3 Ablations, including the failures

Every mechanism was landed under a keep-only-if-faster discipline; the negative results delimit the design space as sharply as the positives.

- **Kernel tiers:** the balanced shuffle-merge took the flagship DNF tree from 130 $\rightarrow$ 57 *ns*; the register-resident rewrite of that kernel halved a 2-way AND (49 $\rightarrow$ 24.8 *ns*, ahead of Roaring’s 33);
- **fan-in promotion:** 4-way sparse OR 203 $\rightarrow$ 54.9 *ns* with the guard shapes (AND of 3-way OR-groups) *unchanged* — the fixed-256 rule, measured, sacrifices one side or the other;
- **fold order:** sparse 8-way DIFF 85 $\rightarrow$ 72 *ns* (partner-major), dense OR unchanged only because the footprint gate routes it key-major — forcing partner-major there costs 12%;
- **hole-punching, automatic:** whole-corpus 3.10 $\rightarrow$ 2.63 s (-15.3% , $p < 0.05$); selective trees 33 $\times$;
- **in-place serialization:** the profiling that motivated it showed DIFF’s fold already *beating* Roaring while the end-to-end number lost — the entire gap was the output copy Roaring never pays; eliminating it flipped the cells and sped up every large-result operation (dense 2-way OR 7.2 $\rightarrow$ 5.1 *ns*);
- **branchlessness, selectively:** table-compaction beats the per-hit emit loop 2.2 $\times$ at dense overlap and is gated to cost nothing at sparse overlap; but a branchless scalar union merge *regressed* 72 $\rightarrow$ 91 *ns* (the predictable branch beats unconditional work), a deferred vector-mask accumulation regressed DIFF 83 $\rightarrow$ 97 *ns* (loop-carried dependency), and an adversarial-vs-predictable input sweep showed the merge advance branches cost *zero* — branchless transformations pay only where decisions are dense and data-dependent;
- **rejected candidates:** window pre-clipping (redundant with galloping), eager bitmap intermediates (wins flat unions, loses tree-consumers), partner-major intersection (linear-scaling control shows nothing to win, and it would weaken the clamps).

10 DEPLOYMENT

FROSTBIT is the foundational bitmap structure for all boolean filtering at Exa, a production web-scale search engine: every filter predicate — domain, language, date, category, and their arbitrary boolean combinations — compiles to a FROSTBIT expression evaluated against frozen per-segment bitmaps served from memory maps. The properties this paper measures are the properties that matter operationally: plans are built once per query and replayed across segments; the zero-allocation hot path removes allocator-induced tail latency; and hole-punching converts the dominant production pattern — one narrow conjunct against wide disjunctions — from the worst case into the best. The frozen (read-only) model is not a limitation in this architecture but the enabling assumption: segments are immutable artifacts of the indexing pipeline, so whole-plan static analysis is sound by construction.

11 CONCLUSION

FROSTBIT demonstrates that after a decade of kernel optimization, the remaining factor of 2–15× in boolean bitmap filtering was hiding in the scaffolding, and that the way to claim it is an analysis pass strong enough to make execution trivial: pre-compute every output, prove every capacity, choose every algorithm and fold order from calibrated cost models, prune every dead container, and pre-allocate every byte — then let a decision-free executor stream the provable minimum. The library, its differential test suites, its benchmark corpus, and the per-tree audit tooling are open source.

REFERENCES

- [1] S. Chambi, D. Lemire, O. Kaser, R. Godin. Better bitmap performance with Roaring bitmaps. *Software: Practice and Experience* 46(5), 2016.
- [2] D. Lemire et al. Roaring Bitmaps: Implementation of an Optimized Software Library. *Software: Practice and Experience* 48(4), 2018.
- [3] K. Wu, E. J. Otoo, A. Shoshani. Optimizing bitmap indices with efficient compression. *ACM TODS* 31(1), 2006.
- [4] B. Schlegel, T. Willhalm, W. Lehner. Fast sorted-set intersection using SIMD instructions. *ADMS@VLDB*, 2011.
- [5] H. Inoue, M. Ohara, K. Taura. Faster set intersection with SIMD instructions by reducing branch mispredictions. *PVLDB* 8(3), 2014.
- [6] D. Lemire, L. Boytsov, N. Kurz. SIMD compression and the intersection of sorted integers. *Software: Practice and Experience* 46(6), 2016.
- [7] S. Han, L. Zou, J. X. Yu. Speeding up set intersections in graph algorithms using SIMD instructions. *SIGMOD*, 2018.
- [8] J. Zhang, Y. Lu, D. G. Spampinato, F. Franchetti. FESIA: A fast and SIMD-efficient set intersection approach on modern CPUs. *ICDE*, 2020.
- [9] S. Kim et al. List intersection for web search: algorithms, cost models, and optimizations. *PVLDB* 12(1), 2019.
- [10] E. D. Demaine, A. López-Ortiz, J. I. Munro. Adaptive set intersections, unions, and differences. *SODA*, 2000.
- [11] B. Goodwin et al. BitFunnel: Revisiting signatures for search. *SIGIR*, 2017.